

CODE SPORT



Sandya Mannarswamy

In this month's column, we continue our discussion on natural language processing.

For the past few months, we have been discussing information retrieval and natural language processing, as well as the algorithms associated with them. This month, we continue our discussion on natural language processing (NLP) and look at how NLP can be applied in the field of software engineering. Given one or many text documents, NLP techniques can be applied to extract information from the text documents. The software engineering (SE) lifecycle gives rise to a number of textual documents, to which NLP can be applied.

So what are the software artifacts that arise in SE? During the requirements phase, a requirements document is an important textual artifact. This specifies the expected behaviour of the software product being designed, in terms of its functionality, user interface, performance, etc. It is important that the requirements being specified are clear and unambiguous, since during product delivery, customers would like to confirm that the delivered product meets all their specified requirements.

Having vague ambiguous requirements can hamper requirement verification. So text analysis techniques can be applied to the requirements document to determine whether there are any ambiguous or vague statements. For instance, consider a statement like, "Servicing of user requests should be fast, and request waiting time should be low." This statement is ambiguous since it is not clear what exactly the customer's expectations of 'fast service' or 'low waiting time' may be. NLP tools can detect such ambiguous requirements. It is also important that there are no logical inconsistencies in the requirements. For instance, a requirement that "Login names should allow a maximum of 16 characters," and that "The login database will have a field for login names which is 8 characters wide," conflict with each other. While the user interface allows up to a maximum of 16 characters, the backend login database will support fewer characters, which is inconsistent with the earlier requirement. Though currently such inconsistent requirements are flagged by human inspection, it is possible to design text analysis tools to detect them.

The software design phase also produces a number of SE artifacts such as the design document, design models in the form of UML documents, etc, which also can be mined for information. Design documents can be analysed to generate automatic test cases in order to test the final product. During the development and maintenance phases, a number of textual artifacts are generated. Source code itself can be considered as a textual document. Apart from source code, source code control system logs such as SVN/ GIT logs, Bugzilla defect reports, developers' mailing lists, field reports, crash reports, etc, are the various SE artifacts to which text mining can be applied.

Various types of text analysis techniques can be applied to SE artifacts. One popular method is duplicate or similar document detection. This technique can be applied to find out duplicate bug reports in bug tracking systems. A variation of this technique can be applied to code clones and copy-and-paste snippets.

Automatic summarisation is another popular technique in NLP. These techniques try to generate a summary of a given document by looking for the key points contained in it. There are two approaches to automatic summarisation. One is known as 'extractive summarisation', using which key phrases and sentences in the given document are extracted and put back together to provide a summary of the document. The other is the 'abstractive summarisation' technique, which is used to build an internal semantic representation of the given document, from which key concepts are extracted, and a summary generated using natural language understanding.

The abstractive summarisation technique is close to how humans would summarise a given document. Typically, we would proceed by building a knowledge representation of the document in our minds and then using our own words to provide a summary of the key concepts. Abstractive summarisation is obviously more complex than extractive summarisation, but yields better summaries.

Coming to SE artifacts, automatic summarisation techniques can be applied to generate large bug reports. They can also be applied to generate high level comments